

PROJE 07 / 20 · ÇALIŞAN TEMEL SÜRÜM

İzole Kod Çalıştırma Laboratuvarı

JavaScript çözümünü Docker içinde kaynak sınırlarıyla çalıştırıp sonuç geçmişini sakla.

Docker gerekli · varsayılan kapalı

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 07 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

İzole Kod Çalıştırma Laboratuvarı

JavaScript çözümünü Docker içinde kaynak sınırlarıyla çalıştırıp sonuç geçmişini sakla.

Docker gerekli · varsayılan kapalı

Çalıştırıcı kapalı: README içindeki Docker kurulumu gereklidir. Kod editörü ve geçmiş ekranı kullanılabilir.

İki sayının toplamı

function solve(a,b) fonksiyonunu yaz. Sonuç sayı olmalı.

JavaScript kodu

```
function solve(a,b) {  
  return a + b;  
}
```

Çalıştırma geçmişi

Henüz gönderim yok.

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Kod editörü ve örnek problem · Kalıcı gönderim geçmişi · Sıralı iş kuyruğu · Ağsız ve salt okunur container · Süre/bellek/çıkış sınırı

Bu sürümde olmayanlar

Docker bu teslim ortamında bulunmadığından gerçek container çalıştırma yolu burada doğrulanmadı. Varsayılan arayüz ve API kapalı runner durumunu gösterir. Tek problem vardır. Aynı süreçteki test harness kötü niyetli kod için adil/yüksek güvenli judge değildir; yalnız yerel eğitim içindir.

DENİZ YARDIMCI · FULL-STACK LAB · 23.09.2026

1

02 / Backend ve veri modeli

API işi SQLite'a queued yazar; worker tek iş alır ve geçici main.js üretir. docker run ağ kapalı, 64 MB bellek, PID sınırı ve nobody kullanıcı ile çağrılır. 8 saniye veya 8 KB çıktı sınırında container sonlandırılır. finally geçici dosyaları temizler. Docker servisini API container içine socket mount ederek açmak bu paketin çalışma modeli değildir.

Gerçek SQL şeması

```
CREATE TABLE submissions(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),code TEXT,status TEXT,output TEXT,created_at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/problems	Gövde yok	200
GET /api/submissions	Gövde yok	200
POST /api/submissions	JSON: code	202

Somut istek örneği

```
POST /api/submissions
{"code":"function solve(a,b){ return a+b; }"}
202 {"id":"...", "status":"queued"}
Runner kapalıysa 503
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Runner kapalıyken arayüz durumu açıkça göstermeli.
- Kod gönderme 503 ve kurulum mesajı döndürmeli.
- Kapalı runner işi queued gibi kaydetmemeli.
- Docker açık yol yerelde ayrıca doğrulanmalı: başarılı çözüm, yanlış sonuç ve süre aşımı.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const problems=await get('/problems');assert.equal(problems.tasks[0].id,'sum');assert.equal(problems.enabled,false);
await post('/submissions',{code:'function solve(a,b){return a+b;}'},503);
assert.equal((await get('/submissions')).length,0);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

Docker çalıştırıcı kapalı yolu test edildi; gerçek container çalıştırma yerelde ek kontrol gerektirir.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Önce runner kapalı durumunu ve 503 mesajını doğrula. README'deki Docker image indirme ve ortam değişkeni adımlarından sonra yerelde örnek çözümü çalıştır.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Container çalışmasını kendi makinenizde doğrula
- 2. Worker'ı ayrı düşük yetkili hizmete ayır
- 3. Test harness sonuç protokolünü güçlendir
- 4. Üretim için ek izolasyon katmanı ve tehdit modeli tasarla

Anlamalı commit sırası

```
git add .
git commit -m "chore: add code-judge baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Container tek başına yeterli bir güvenlik sınırı mıdır? Bir gönderim işleyicisi neden ana API sürecinden ayrılmalıdır?

Özel kurulum / ilerleme notu

Docker Engine çalışır durumda olmalı. Önce `docker pull node:24-alpine` çalıştır. `.env` dosyasına `ENABLE\_RUNNER=1` yaz; `node --env-file=.env server/index.js --dev` ile başlat. Varsayılan runner kapalıdır. Bu adaptörün gerçek container yolu teslim ortamında test edilmedi. Yerel denemede accepted, failed ve limit durumlarını ayrı doğrula.